RR-Net: 3-D Roof Reconstruction From Airborne LiDAR Point Clouds via Edge Segmentation and Wireframe Generation

Yike Xu, *Graduate Student Member, IEEE*, Li Cao, Shengling Geng, and Xiaoping Liu

***Abstract*—Reconstruction of 3-D roof from airborne light detection and ranging (LiDAR) point clouds is an important task in the field of remote sensing and photogrammetry. Due to the high noise, large volume, and structural complexity inherent in LiDAR point clouds, traditional point cloud processing methods have limited performance in terms of accuracy and robustness. Most existing methods rely on corner point detection and edge prediction to extract and reconstruct roof structures. However, corner points are often sparsely distributed and difficult to locate precisely, resulting in noticeable geometric deviations and incomplete structural reconstruction. Therefore, generating a high-quality 3-D roof from complex LiDAR point clouds is still a challenging problem to be solved. To address these issues, we propose a novel roof reconstruction method RR-Net that combines edge segmentation and wireframe generation. First, an innovative unified point cloud edge segmentation network was developed, achieving integrated edge detection, segmentation, and denoising. Second, based on the segmentation results from this network, an efficient and robust pipeline for wireframe generation and roof reconstruction was designed, enabling automated transformation from airborne LiDAR point clouds to roof models. Extensive experiments on the Building dataset demonstrate that RR-Net can efficiently and accurately reconstruct roofs from airborne LiDAR point clouds. RR-Net achieves 95.7% accuracy in edge detection and reduces the Chamfer distance (CD) error of the wireframe to 0.021. The results of RR-Net are publicly available at https://yecoxu.github.io/publications/RR-Net/**

***Index Terms*—Edge segmentation, light detection and ranging (LiDAR), point clouds, roof reconstruction, wireframe generation.**

I. Introduction

With the rapid development of airborne light detection and ranging (LiDAR) technology, its ability to efficiently capture 3-D point cloud data in urban environments has significantly improved. Airborne LiDAR point clouds have significant characteristics of high precision and density, which can accurately describe the geometric features of object surfaces [1]. However, airborne LiDAR point clouds often contain vast amounts of information, resulting in high computational complexity and long processing times, thus placing high demands on computational resources and algorithm efficiency. In contrast, 3-D wireframe, as a lightweight data representation, can comprehensively express the shape and structural information of 3-D objects with a lower data volume, offering advantages in storage efficiency and reduced computational complexity [2]. Additionally, 3-D wireframe can be conveniently converted into 3-D mesh models, making them highly valuable in practical applications such as urban modeling, terrain surveying, and digital twins [3], [4], [5], [6].

Finding an efficient and concise representation for large-scale airborne LiDAR point cloud data has been an important research direction in point cloud processing [7], [8]. Previous studies were primarily inspired by 2-D image line segment detection methods, attempting to extend similar concepts into 3-D space by detecting 3-D line segments within point clouds to extract key structural features [9], [10]. The detection of 3-D line segments typically relies on formal geometric rules, such as the intersection of two nonparallel planes [11]. Although such methods offer advantages in accurately identifying linear structures, they also exhibit significant limitations. On the one hand, discrete line segments are inadequate to fully represent the continuous geometric features of complex objects. On the other hand, relying solely on line segments is insufficient for effectively capturing the topological structure and semantic information of buildings, failing to meet the demands of practical applications.

In recent years, deep learning has achieved continuous breakthroughs in the field of 3-D computer vision. In particular, pioneering methods such as PointNet [12] and PointNet++ [13] have opened up a new research paradigm for efficient point cloud processing. Meanwhile, the release of high-quality datasets such as Building3D [14] and Semantic3d [15] provides a solid data foundation for point cloud processing tasks on building and roof. The synergistic development of the above technologies significantly improves the accuracy and efficiency of point cloud processing, making roof reconstruction and wireframe generation a hot research topic.

Existing deep learning-based wireframe generation methods, although performing well under ideal conditions, are mainly trained and tested on clean point clouds, which are difficult to be directly applied to real LiDAR point clouds.

Received 1 July 2025; revised 15 September 2025; accepted 20 October 2025. Date of publication 27 October 2025; date of current version 5 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62277014, in part by Anhui Key Research and Development Program under Grant 202104e11020006, and in part by Key Research and Development and Transformation Plan Projects of Qinghai under Grant 2025-QY-215. *(Corresponding author: Li Cao.)*

Yike Xu, Li Cao, and Xiaoping Liu are with the School of Computer and Information Science, Hefei University of Technology, Hefei 230006, China (e-mail: yecoxu@mail.hfut.edu.cn; lcao@hfut.edu.cn; liu@hfut.edu.cn).

Shengling Geng is with the School of Computer Science, Qinghai Normal University, Xining 810000, China (e-mail: geng_sl@126.com).

Digital Object Identifier 10.1109/TGRS.2025.3625669

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

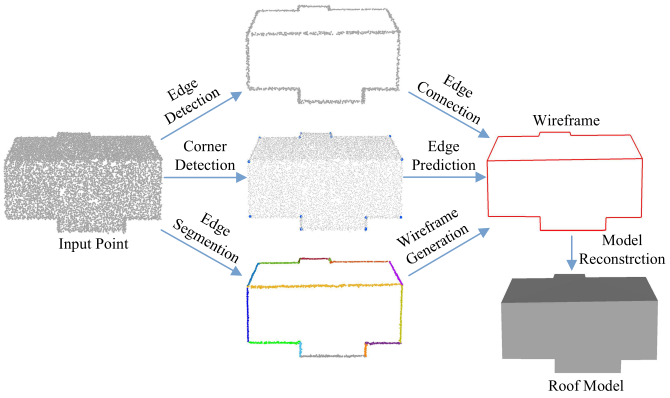


Fig. 1. Different pipeline for wireframe generation and roof reconstruction.

Airborne LiDAR point clouds usually suffer from uneven density, noise interference, missing occlusions, and blurred boundaries, which seriously affect the applicability of existing methods. As depicted in Fig. 1, existing methods primarily rely on corner point detection, edge detection, and edge prediction [16] for wireframe generation and roof reconstruction. Although two-stage methods based on corner point detection and edge prediction improve noise robustness to some extent, their performance is still limited by error accumulation effects. Specifically, the errors in corner point detection are transmitted to the edge prediction, thus affecting the overall wireframe accuracy and topology.

Our key insight is that effective denoising and segmentation of edges can lead to more accurate wireframe generation, thereby significantly improving the overall quality of roof reconstruction. Building upon these observations, we propose RR-Net, a novel framework for efficient building roof structure reconstruction from airborne LiDAR point clouds. As depicted in Figs. 2 and 3, we first preprocess the airborne LiDAR point clouds to generate input point clouds with complete labels. Then, we design an edge-aware segmentation network architecture for edge classification, segmentation, and denoising of point clouds. This network effectively identifies and optimizes edge features, thereby enhancing the accuracy of subsequent modeling. Building upon this foundation, we further developed a comprehensive wireframe generation and roof reconstruction pipeline, including corner reconnection, line fitting, and planar reconstruction. Through precise edge extraction and refinement, our approach produces more stable roof models that better conform to authentic geometric structures.

The main contributions can be summarized as follows.

1) We propose RR-Net, a novel method for reconstructing 3-D roof from airborne LiDAR point clouds via edge segmentation and wireframe generation.
2) We propose an edge segmentation module that integrates edge detection, edge point segmentation, and denoising to effectively deal with noise interference in complex point clouds.
3) We design a roof reconstruction module that effectively leverages edge segmentation results to efficiently and accurately recover roof structures.
4) We provide the Building3D-edge dataset with multilevel annotations, including edges, segmentation, and noise labels, further expanding the scope of geometric deep learning on airborne LiDAR point clouds.

II. Related Work

Roof extraction and reconstruction have been extensively studied in 2-D images. In contrast, research on 3-D point clouds faces greater challenges due to data sparsity, noise interference, and the complexity of high-dimensional spatial features. This section reviews existing work from three perspectives: edge point detection, wireframe generation, and roof reconstruction.

A. Edge Point Detection

As a critical feature of 3-D point clouds, edge point detection has been widely investigated [17]. In the processing of unstructured point cloud data, edge detection typically serves as the primary step for achieving high-quality surface reconstruction [18], [19] and precise geometric processing [20]. Recent research trends indicate heightened interest in edge point detection, particularly for remote sensing and computer-aided design (CAD) applications.

In remote sensing, large-scale outdoor scene point clouds [15] are massive, unstructured, and spatially irregular, making edge detection particularly challenging. Existing methods estimate edge confidence using local geometric features such as curvature, normals, and gradients [21], but struggle to capture global correlations. In contrast, learning-based methods offer stronger global feature perception. Hackel et al. [22] used a binary classifier on local geometry to predict contour probabilities. Di et al. [23] formulated the edge extraction task as a Bayesian inference problem. Himeur et al. [24] used scale-space matrices to encode local geometric features around each point, subsequently detecting edge points through a lightweight neural network architecture. Zang et al. [25] proposed an end-to-end framework with vertex detection and proposal discrimination tailored for large-scale edge point detection.

In the field of CAD, current research primarily utilizes the ABC benchmark dataset [26]. Huang et al. [27] proposed an edge-aware point cloud resampling algorithm that effectively preserves geometric edge features while generating noise-free normal vectors. Currently, the data-driven approach has become the mainstream research, which achieves edge detection by learning latent features from training data. EC-Net [28] innovatively incorporates edge awareness into the upsampling process, though its performance is constrained by the prerequisite of partitioning point clouds into regions. PIE-Net [29] employs an end-to-end architecture that implements edge detection via a binary classification mechanism. Within deep learning-based edge detection, modeling the problem as a per-point binary classification task is the most practical approach [18], [30]. Zhao et al. [31] proposed a multitask neural network architecture capable of identifying points near sharp features and predicting displacement vectors pointing toward local sharp structures. Jiao et al. [32] introduced a feature extraction method based on intrinsic neighborhood shape descriptors, enabling efficient detection of sharp features from point clouds.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

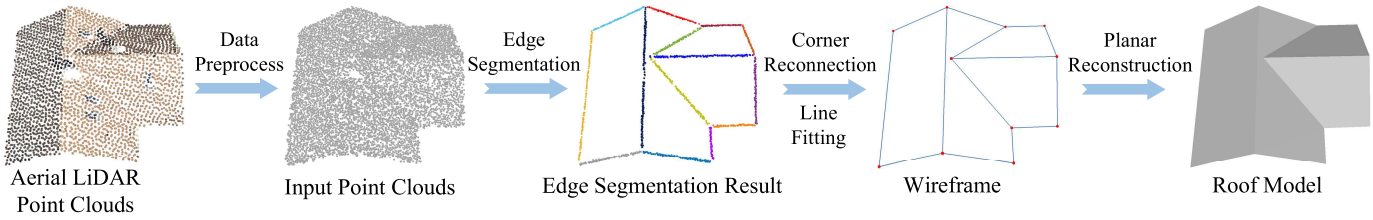


Fig. 2. Overall pipeline of RR-Net.

B. 3-D Wireframe Generation

For large-scale point clouds, prior studies have mainly focused on line segment extraction rather than complete wireframe generation. Lin et al. [11] extracted line segments based on local planar features, while Zhang et al. [33] proposed a parameter space learning framework with a guided mechanism to optimize contour distribution. Tian et al. [10] projected the 3-D structure detection problem into 2-D to infer 3-D lines via planar boundary fitting. Xin et al. [34] introduced a weighted centroid displacement and L_1-median optimization for precise feature point detection. To reduce computation, some methods adapted 2-D edge detection by projecting 3-D regions onto image planes, extracting contours, and reprojecting them back into 3-D [9].

Compared to discrete line extraction, generating complete wireframes from large-scale point clouds is significantly more challenging. To address this, Jiang et al. [4] proposed a multistage framework involving corner detection, edge prediction, and refinement. While deep learning holds promise, most studies still rely on CAD models due to the computational burden of large-scale data. PC2WF [2] separates wireframe generation into corner detection and connection, producing sparse, linear structures. NerVE [35] fits curves using structured edge detection and curve classification. Yang et al. [36] incorporated point cloud completion to address sparsity and improve structural accuracy. Huang et al. [37] proposed an end-to-end framework that directly regresses edge parameters. Liu et al. [38] employed a conditional diffusion model for wireframe generation under noise, while Liu et al. [39] used 2-D heatmaps and a Transformer with height embeddings to lift corners into 3-D and predict edge connections. It is worth noting that, compared with direct point cloud processing, line cloud-based modeling demonstrates improved consistency in generating wireframe structures [40].

C. Roof Reconstruction

Previous methods for building roof reconstruction often relied on combinations of predefined simple parametric shapes. Verma et al. [41] automatically identified and estimated such shapes to model complex roof structures. Henn et al. [42] used a model-driven approach that classifies roof types via support vector machines and estimates parameters with random sample consensus (RANSAC). Huang et al. [43] introduced a generative method using Markov chain Monte Carlo to optimize the selection and composition of shapes from a primitive library. Xiong et al. [44] modeled roofs using topological elements and shape constraints to achieve automated reconstruction. However, due to the diversity of roof forms, a fixed set of geometric shapes cannot fully represent all topological variations.

Data-driven methods are typically divided into two stages. The first stage involves extracting roof plane primitives from LiDAR point clouds using a plane segmentation algorithm [45]. The second stage reconstructs the roof model by analyzing the original topology. Cao et al. [46] proposed a parallelized 3-D building roof automatic reconstruction framework based on spatial databases, which achieves efficient processing by integrating data- and model-driven approaches. Chen et al. [47] proposed a building roof detection and reconstruction method based on a multiscale grid. Recently, with the development of deep learning technology, roof reconstruction methods based on deep learning have been proposed. The introduction of PC2WF [2] provides a new idea for roof reconstruction, combining corner point detection and edge prediction. Since roof frameworks are typically composed of straight line segments, the framework structure can be effectively determined through the connection of corner points [48], [49]. Li et al. [50] proposed Point2Roof, which uses PointNet++ to extract features, detect and refine corner points, and predict their connectivity to generate roof models. However, accurately localizing corners in sparse and complex LiDAR point clouds remains challenging.

III. Method

For a given airborne LiDAR point cloud, our goal is to reconstruct a complete roof model. This process primarily involves three key steps. First, due to the lack of datasets with edge segmentation annotations, we preprocess the Building3D dataset to create the Building3D-edge dataset. Second, we aim to accurately segment the edges of the point cloud. Finally, we generate a wireframe from the segmented point cloud edges and further perform roof reconstruction.

A. Data Preprocess

Building3D dataset [14] is a large-scale airborne LiDAR point cloud dataset that spans 16 cities across Estonia and includes approximately 160 000 buildings. The original dataset exhibited issues such as low density, limited sample size, and high shape similarity. To ensure the complexity and representativeness of the data, we performed an initial filtering process, primarily removing samples with overly simple shapes (e.g., regular rectangles) and those with fewer than 1000 points.

The Building3D dataset provides a substantial amount of rooftop point cloud data with wireframe annotations. Unlike

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

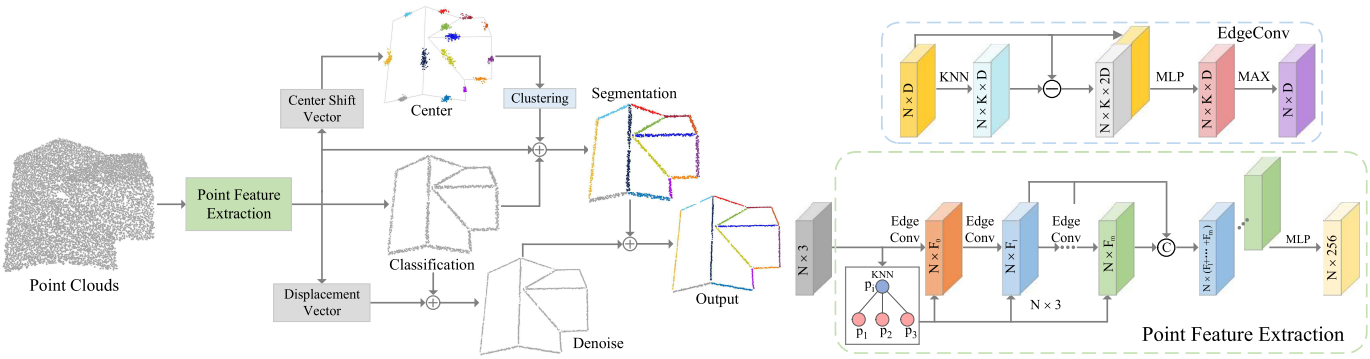


Fig. 3. Overall network framework of RR-Net.

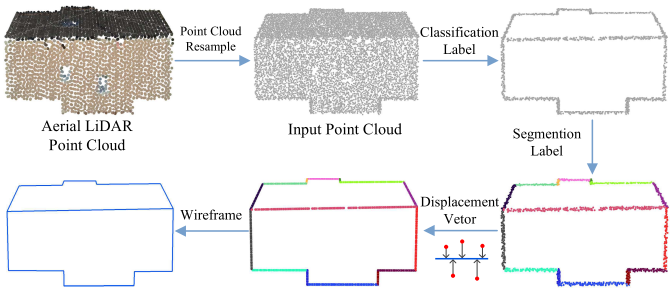


Fig. 4. Building3D-edge dataset annotation process.

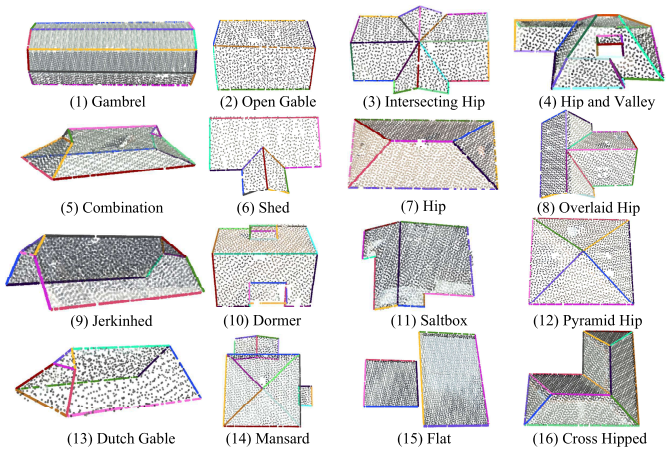


Fig. 5. Sixteen typical roof types included in Building3D-edge.

the ABC dataset [26], which provides explicit edge point annotations, the Building3D dataset does not annotate edge points. Furthermore, the Building3D dataset consists of raw point cloud scan results, meaning that the point cloud data contains noise and exhibits uneven density distribution. Previous edge detection and wireframe generation methods are primarily trained and tested on the clean point clouds provided by the ABC dataset. Therefore, the applicability of previous methods on the Building3D dataset is limited. Therefore, we preprocessed the Building3D dataset to generate the labeled Building3D-edge dataset.

As shown in Fig. 4, the annotations include point cloud classification, segmentation, and displacement vector. Classification divides the point cloud into edge points and nonedge points. Segmentation further divides the edge points into multiple local regions based on classification. The displacement vector represents the distance from each edge point to the nearest wireframe.

Roofs are typically composed of multiple local planar regions. Therefore, during point cloud resampling, it is essential to preserve the geometric integrity of the point cloud while avoiding the introduction of artificial noise or distortion. Specifically, the resample process begins with a coarse planar segmentation of the roof point cloud using the RANSAC algorithm. For each planar region, a local plane is fit for each point's neighborhood using principal component analysis (PCA), and new points are generated through random interpolation on these fit planes. Next, the resampled points are projected onto their nearest line segments. We set a distance threshold D. For each projected point, if the distance to the line segment is less than D, it is labeled as an edge point; otherwise, it is labeled as a nonedge point. Meanwhile, the association between each edge point and its corresponding nearest line segment is recorded to enable precise segmentation and annotation.

The Building3D dataset encompasses a wide variety of roof types, demonstrating strong diversity and representativeness. Although we further filtered the dataset to better align with the requirements of edge segmentation and reconstruction tasks, Building3D-edge still retains a rich collection of roof types, thereby ensuring the generalizability and robustness of the experimental results. Fig. 5 illustrates 16 representative roof types, such as gambrel, hip, flat, and shed. Overall, Building3D-Edge essentially covers the roof types commonly found in urban buildings.

B. Edge Segmentation Network

Given a 3-D point cloud $\mathbf{P} = \{\mathbf{p}_i \in \mathbb{R}^{N\times 3}\}$, where $i = \{1, \ldots, N\}$, the goal of the edge segmentation network is to produce a denoised edge segmentation result. Let $\mathbf{B} = \{\mathbf{b_i} \in [0,1]^2\}$ denote the binary edge points classification, which classifies a point as an edge point or not. $\mathbf{S} = \{\mathbf{s_i} \in [0,1]^K\}$ represents the association between edge points and line segments, and $\mathbf{O} = \{\mathbf{o_i} \in \mathbb{R}^3\}$ indicates the projection distances. We set $K = 64$. $\mathbf{s_i} \in \{0,1\}$ is a binary mask. $\mathbf{s_i} = 1$ if point $\mathbf{p}_i$ belongs to a line segment and $\mathbf{s_i} = 0$ otherwise.

As shown in Fig. 3, we first design a point feature extraction module that stacks multiple layers of EdgeConv [51] to efficiently capture local geometric features of the point

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

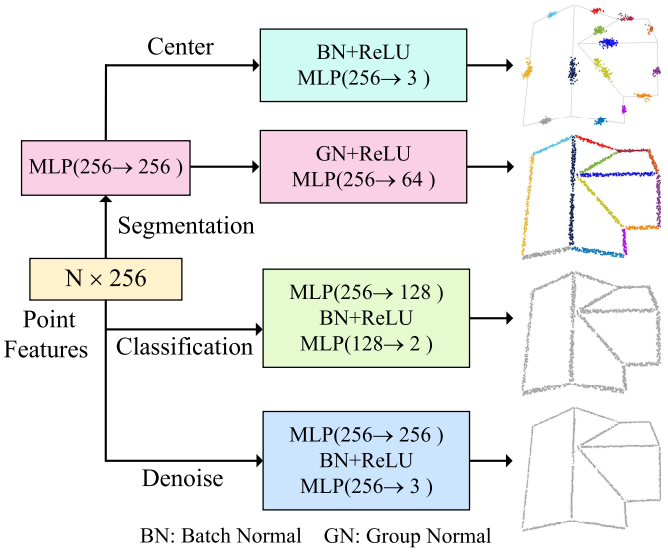


Fig. 6. Details of the classification, segmentation, and denoise branches.

cloud. Additionally, dense connections are employed to fuse the outputs of different EdgeConv layers, thereby enhancing the capability of global feature extraction. In each EdgeConv layer, we adopt a consistent K-nearest neighbor (KNN) strategy to ensure the consistency and stability of local feature extraction. Subsequently, we design three dedicated branches to perform edge classification, segmentation, and denoising. Fig. 6 presents the detailed network design of each branch.

1) Classification: For edge point classification, we formulate it as a binary classification problem, predicting the probability distribution of each point being an edge or nonedge point. Previous edge detection methods for point clouds typically focus solely on binary classification of points, followed by global geometric modeling. Such approaches often fail to capture local edge characteristics, leading to significant prediction bias.

2) Segmentation: Inspired by advances in point cloud instance segmentation [52], we propose a novel edge segmentation strategy that incorporates local centroid features to enhance boundary awareness. The local center feature is represented by a displacement vector pointing from each edge point to its corresponding local center. Specifically, we partition the edge points into multiple independent regions and guide each point to cluster toward the center of its associated region. Each independent local region corresponds to a line segment in the roof wireframe.

Therefore, the edge segmentation branch consists of two components. The first component employs a multilayer perceptron (MLP) to decode edge segmentation features from the extracted point cloud features. The second component further regresses the displacement vectors from edge points to their corresponding local centers based on the segmentation features, guiding the edge points to aggregate toward the same local region. The second component further regresses the displacement vectors from edge points to their corresponding local centers based on the segmentation features, guiding the edge points to aggregate toward the same local region. Relying solely on segmentation results may be difficult to achieve accurate segmentation, especially at edge intersections. By introducing a center displacement module, edge points can adaptively converge toward their corresponding central positions, enabling effective clustering of the point cloud. This strategy allows points with identical segmentation labels to naturally cluster into the same group, significantly improving edge segmentation accuracy and enhancing the ability to distinguish complex edge structures, thereby producing clearer and more coherent edge segmentation results. Finally, the segmentation point map $\hat{S}$ and the center shift vector $\hat{\mathbf{V}} = \{\hat{\mathbf{v}}_\mathbf{i} \in \mathbb{R}^3\}$ are decoded from point features.

3) Denoise: The key objective of edge denoising is to guide edge points toward their corresponding line segments to enable more accurate edge reconstruction. We employ a two-layer MLP to predict per-point displacement vectors. The predict displacement vector is denoted as $\hat{\mathbf{O}} = \{\hat{\mathbf{o}}_\mathbf{i} \in \mathbb{R}^3\}$. This displacement vector $\hat{\mathbf{o}}_\mathbf{i}$ represents the repositioning direction from each edge point to the nearest line segment, effectively eliminating noise and offset while maintaining the overall structure. Through this process, edge points are adaptively refined to better align with the ideal geometric contours, resulting in smoother segmentation outcomes that more closely conform to the true geometric structure.

C. Loss Functions

The overall loss function consists of classification loss L_c, segmentation loss L_s, center shift loss L_{cen}, and predicted displacement vector loss L_{dis}. The total loss L is calculated as

$$L = \alpha_{\text{cls}} L_{\text{cls}} + \alpha_{\text{seg}} L_{\text{seg}} + \alpha_{\text{cen}} L_{\text{cen}} + \alpha_{\text{dis}} L_{\text{dis}}. \tag{1}$$

In our implementation, $\alpha_{\text{cls}} = 1$, $\alpha_{\text{seg}} = 1$, $\alpha_{\text{cen}} = 0.1$, and $\alpha_{\text{dis}} = 0.01$ for the first 15 epochs. For the following 25 epochs, we adjusted the parameters to $\alpha_{\text{cls}} = 0.01$, $\alpha_{\text{seg}} = 0.01$, $\alpha_{\text{cen}} = 0.5$, and $\alpha_{\text{dis}} = 1$. We guide the network to focus on different tasks at different stages of training. In the early phase, the optimization is primarily directed toward improving classification and segmentation performance. In the later stages, the network is progressively encouraged to learn denoising and center offset prediction. This strategy is based on the observation that the accurate prediction of displacement vectors relies on relatively reliable classification results; without reasonably accurate classification, the predicted offsets lack practical significance.

1) Classification Loss: Edge points account for only a small fraction of the entire point cloud. To address the issue of class imbalance, we adopt focal loss [53] as the binary classification loss for the point cloud, which effectively enhances the model ability to capture critical edge features

$$\begin{aligned} L_{\text{cls}} &= -(1-p_t)^{\gamma} \log(p_t) \\ p_t &= \begin{cases} p & \text{if} \quad \mathbf{b}_\mathbf{i} = 1 \\ 1-p & \text{otherwise} \end{cases} \end{aligned} \tag{2}$$

where p is the predicted probability of the edge point, $\mathbf{b}_\mathbf{i}$ is the true label, and γ is the parameter that controls the strength of weight attenuation based on sample difficulty.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

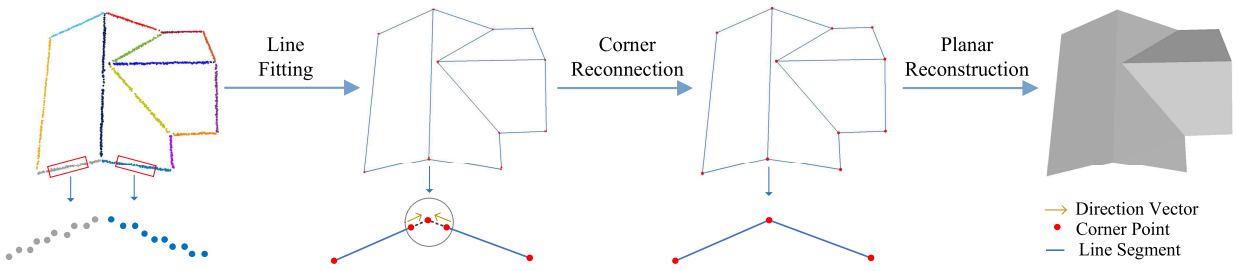


Fig. 7. Roof reconstruction pipeline based on edge segmentation results.

2) Segmentation Loss: The segmentation loss is designed to pull edge points from the same region closer together while pushing those from different regions apart, thereby forming clearer and more structured clusters [54]. Given a triplet of points (a, b, c), triplet loss is defined as

$$\ell_{\text{seg}}(a,b,c) = \max\left(0, \|\mathbf{F}_a - \mathbf{F}_b\|^2 - \|\mathbf{F}_a - \mathbf{F}_c\|^2 + \tau\right) \tag{3}$$

where τ is the margin set to 0.9. $\mathbf{F}_a, \mathbf{F}_b$, and $\mathbf{F}_c$ are the feature vector of points (a, b, c) in the embedding space. Given a triplet set $\mathcal{T}_S$ sampled from each point set $\mathbf{P}$, the segmentation objective sums the loss over triplets

$$L_{\text{seg}} = \sum_{(a,b,c)\in\mathcal{T}_S} \ell_{\text{seg}}(a,b,c). \tag{4}$$

3) Center Shift Loss: The center offset loss aims to encourage edge points within each region to move closer to the centroid of their respective regions. For points belonging to the same region, we constrain their learned offset vectors using an L_1 regression loss to ensure that they point toward the correct center

$$L_{\text{cen}} = \frac{1}{\sum_i \mathbf{s_i}} \sum_i \|\hat{\mathbf{v}}_\mathbf{i} - (\mathbf{c_i} - \mathbf{p_i})\| \cdot \mathbf{s_i} \tag{5}$$

where $\mathbf{c_i}$ is the centroid of the region that point $\mathbf{p_i}$ belongs to

$$\mathbf{c_i} = \frac{1}{N^R_{g(i)}} \sum_{j\in R_{g(i)}} \mathbf{p_j} \tag{6}$$

where $g(i)$ maps points $\mathbf{p_i}$ to the index of its corresponding ground truth (GT) region, which contains $\mathbf{p_i}$, and $N^R_{g(i)}$ is the number of point in region $R_{g(i)}$.

4) Displacement Vector Loss: We used masked mean-squared error loss for displacement vector prediction

$$L_{\text{dis}} = \frac{1}{\sum_{i=1}^N \mathbf{b_i}} \sum_{i=1}^N \mathbf{b_i}(\mathbf{o_i} - \hat{\mathbf{o}}_\mathbf{i})^2 \tag{7}$$

where $\mathbf{o_i}$ is the GT displacement vectors and $\hat{\mathbf{o}}_\mathbf{i}$ is the predicted displacement vectors.

D. Roof Reconstruction

As shown in Fig. 7, the roof reconstruction pipeline consists of line fitting, corner reconnection, and planar reconstruction. A wireframe structure is composed of a set of key points and the connecting edges between them. By accurately determining the spatial coordinates of the points and explicitly defining the topological relationships among them, a complete wireframe can be reconstructed. Accordingly, existing methods typically adopt a pipeline that involves corner point detection followed by connection relationship prediction for wireframe generation. However, such approaches suffer from two critical issues. First, the localization of corner points is prone to coordinate deviations. Second, the prediction of connectivity is susceptible to failure. As illustrated in Fig. 8, both types of errors can severely affect the final reconstruction results. To address the above issues, we propose a complete roof model reconstruction pipeline based on high-precision edge segmentation results.

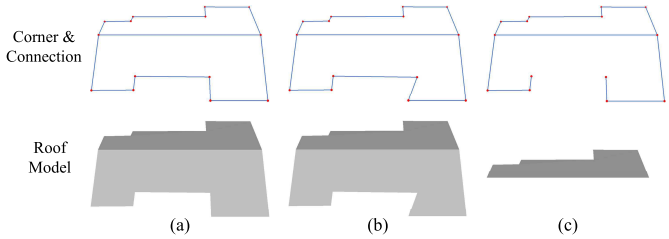


Fig. 8. (a) GT. (b) Corner coordinate deviation. (c) Incorrect connection prediction.

1) Line Fitting: First, the centroid of the point cloud is computed. Then, singular value decomposition (SVD) is applied to extract the direction with the greatest variance in the point cloud, which serves as the direction vector of the fit line. Next, the edge points are projected onto this direction to determine the minimum and maximum projection values, ensuring that the length of the fit line segment covers the entire spatial extent of the point cloud. Finally, two corner points of the fit line are generated along with their connection information.

2) Corner Reconnection: Since line fitting is performed in a piecewise manner, it may lead to discontinuities or disjoint segments between adjacent lines. To improve the overall continuity of the wireframe structure, we utilize the direction vectors of individual segments to guide the reconnection of corner points. Specifically, we compute the intersection point of multiple segment direction vectors within a local region as a candidate corner point. This intersection point more accurately reflects the geometric convergence in real structures, thereby enhancing the accuracy and consistency of wireframe connectivity.

3) Planar Reconstruction: The roof structure is typically composed of multiple interconnected planar surfaces. As depicted in Fig. 9, to efficiently reconstruct the model, we formulate the roof reconstruction task as a graph search problem based on a wireframe representation. In this formulation,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

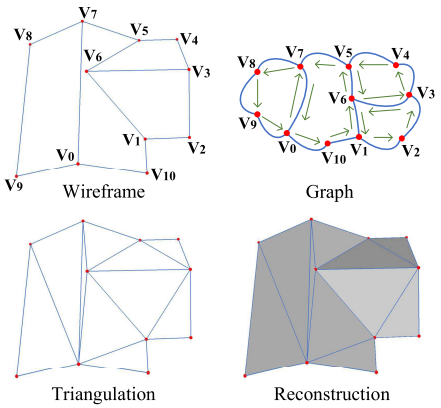


Fig. 9. Roof reconstruction based on converting wireframes into graph.

the corner points of the wireframe are treated as nodes in a graph, and the connections between these corners are modeled as edges, thereby constructing a graph-based representation of the roof. We identify potential roof planes by searching for the shortest closed loop associated with each vertex in the graph. Finally, for each identified plane, we apply Delaunay triangulation to generate the reconstruction model. The algorithmic procedure is presented in Algorithm 1.

Algorithm 1 Roof Reconstruction Algorithm Based on Edge Segmentation

Input: Edge segmentation point cloud P
Output: Reconstructed roof mesh M

Step 1: Line Fitting
Partition P into local subsets $\{P_i\}$;
for *each subset* P_i **do**
  Compute centroid c_i of P_i;
  Perform SVD on $(P_i - c_i)$ to obtain principal direction v_i;
  Project points in P_i onto v_i to get projection values $\{p_{ij}\}$;
  $min \leftarrow \min(\{p_{ij}\})$, $max \leftarrow \max(\{p_{ij}\})$;
  $endpoint_1 \leftarrow c_i + min \cdot v_i$;
  $endpoint_2 \leftarrow c_i + max \cdot v_i$;
  Add $endpoint_1, endpoint_2$ to corner set C;
  Add LineSegment($endpoint_1, endpoint_2$) to line set L;

Step 2: Corner Reconnection
for *each local region* R *of line segments in* L **do**
  $dirs \leftarrow \{segment.direction \mid segment \in R\}$;
  $intersection \leftarrow$ ComputeIntersection($dirs$);
  Update corner set $C \leftarrow C \cup \{intersection\}$;

Step 3: Planar Reconstruction
Construct graph $G(C, L)$ with corners C as nodes and lines L as edges;
for *each vertex* $v \in C$ **do**
  Find shortest closed loop $\mathcal{L}$ in G associated with v;
  Mark $\mathcal{L}$ as candidate plane boundary;
for *each candidate plane boundary* $\mathcal{L}$ **do**
  $plane \leftarrow$ DelaunayTriangulation($\mathcal{L}$);
  $M \leftarrow M \cup plane$;
return M;

IV. Experiment

In this section, we provide a detailed description of the experimental setup and evaluate the performance of the proposed method on three tasks: corner detection, wireframe generation, and roof reconstruction. Extensive experiments are conducted on an airborne LiDAR point clouds dataset to verify the effectiveness and generalizability of our approach.

A. Dataset and Implementation

Our Building3D-edge dataset is built upon the Building3D [14] dataset and covers more than ten types of roof structures, including Flat, Dormer, Mansard, Pyramid Hip, Saltbox, and others. To evaluate the performance of our method, we conducted experiments on the Building3D-edge dataset described in Section III-A. To ensure a fair and consistent comparison with state-of-the-art methods, we incorporated the RoofN3D dataset [55] used by those methods into our experiments.

The number of points N is set to 10 000. The distance threshold D is set to 0.02. The model was trained on a single NVIDIA RTX 4080 GPU with a batch size of 40. The training time for the Building3D-edge dataset was approximately 80 h.

B. Evaluation Metrics

Similar evaluation metrics [35], [50] were used to evaluate the proposed RR-Net. To evaluate the positional accuracy of corner points in the wireframe, we adopt the VD_x, VD_y, VD_z metric, which measures the average positional deviation between the predicted and GT corners on the x-, y-, and z-axes [50]. For evaluation, accuracy (Acc), precision (Prec), and recall (Rec) are used to quantitatively assess edge points detection. To compare the quality of wireframes with other methods, we used the Chamfer distance (CD) and the mean Hausdorff distance (HD) to assess the similarity between the two point sets. CD and HD can be calculated as

$$\mathrm{CD}(X,Y) = \frac{1}{|X|}\sum_{x\in X}\min_{y\in Y}\|x-y\|_2^2 + \frac{1}{|Y|}\sum_{y\in Y}\min_{x\in X}\|y-x\|_2^2 \quad (8)$$

$$\mathrm{HD}(X,Y) = \max\left\{\sup_{x\in X}\inf_{y\in Y} d(x,y), \sup_{y\in Y}\inf_{x\in X} d(x,y)\right\} \quad (9)$$

where X and Y are assumed to be two finite point sets and $d(x, y)$ is the Euclidean distance between points x and y. In addition, to more comprehensively evaluate the runtime efficiency, we introduce the average processing time (Time) as a comparative reference.

For evaluating the quality of the wireframe, we use the wireframe edit distance (WED) [2] to measure the discrepancy between the generated wireframe and GT. WED measures similarity by calculating the minimum cost of edit operations needed to convert the generated wireframe into the GT. The edit operations contributing to the cost include adding and deleting edges, and moving or inserting points.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

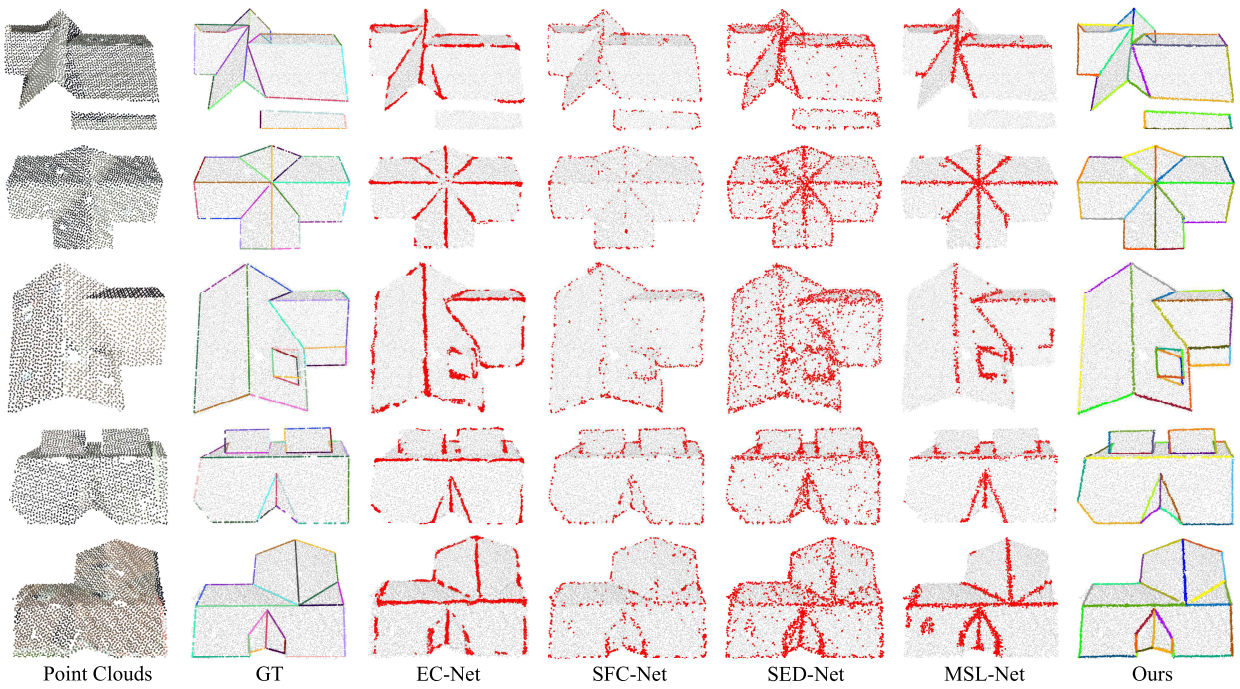


Fig. 10. Comparison experiment results of edge points.

TABLE I
QUANTITATIVE EVALUATION RESULTS OF EDGE POINTS

Method	CD↓	Acc↑	Prec↑	Rec↑	Time(s)↓
EC-Net	0.094	0.879	0.611	0.664	2.859
SFC-Net	0.041	0.894	0.803	0.855	0.753
SED-Net	0.053	0.917	0.823	0.871	0.945
MSL-Net	0.119	0.849	0.751	0.591	18.56
Ours	**0.019**	**0.957**	**0.868**	**0.940**	**0.476**

TABLE II
COMPARISON EXPERIMENT RESULTS OF CORNER POINTS

Dataset	Method	VD_x	VD_y	VD_z	Time(s)↓
RoofN3d	RoofVE	0.382	0.339	0.134	41.45
	Point2Roof	0.098	0.112	0.079	**0.271**
	Ours	**0.056**	**0.068**	**0.039**	0.425
Building3D-edge	Point2Roof	0.139	0.165	0.133	**0.376**
	Ours	**0.062**	**0.077**	**0.046**	0.504

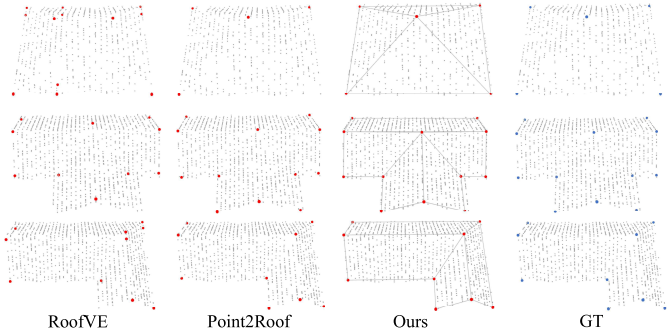


Fig. 11. Comparison experiment results of corner points.

C. Comparative Experiments

In this section, we conduct a detailed comparative analysis of four tasks: edge detection, wireframe generation, corner detection, and roof reconstruction.

1) Edge Detection: We conducted a comprehensive comparison of several advanced learning-based edge detection methods, including EC-Net [28], SFC-Net [31], SED-Net [18], and MSL-Net [32]. The visualized results of edge detection are shown in Fig. 10. EC-Net performs upsampling and region-wise edge detection on point clouds, resulting in denser edge predictions but lower overall accuracy. SFC-Net incorporates a simple denoising module, providing a certain level of noise resistance. However, it requires high edge sharpness in the input data. Since edges in real-world LiDAR point clouds often exhibit smooth transitions, SFC-Net is not sufficiently sensitive to detecting sharp features. SED-Net is capable of detecting more edge points, but its weak noise robustness leads to lower precision and higher recall. MSL-Net detects edges based on variations in normal vectors, enabling effective detection at the junctions of planar patches, but it performs poorly on boundary edges within single surfaces. In contrast, our method achieves higher accuracy in edge detection, effectively performs edge segmentation and denoising, and demonstrates stronger noise robustness and adaptability to complex LiDAR point clouds. The quantitative evaluation is reported in Table I. In terms of runtime efficiency for the edge detection task, our method achieves the shortest average processing time. The longer runtime of MSL-Net is primarily due to its multistage design, which requires segment-wise data processing, whereas the time consumption of EC-Net is mainly attributed to the upsampling process.

RR-Net demonstrates optimal performance on relatively simple roof types such as Open Gable, Box Gable, Flat,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

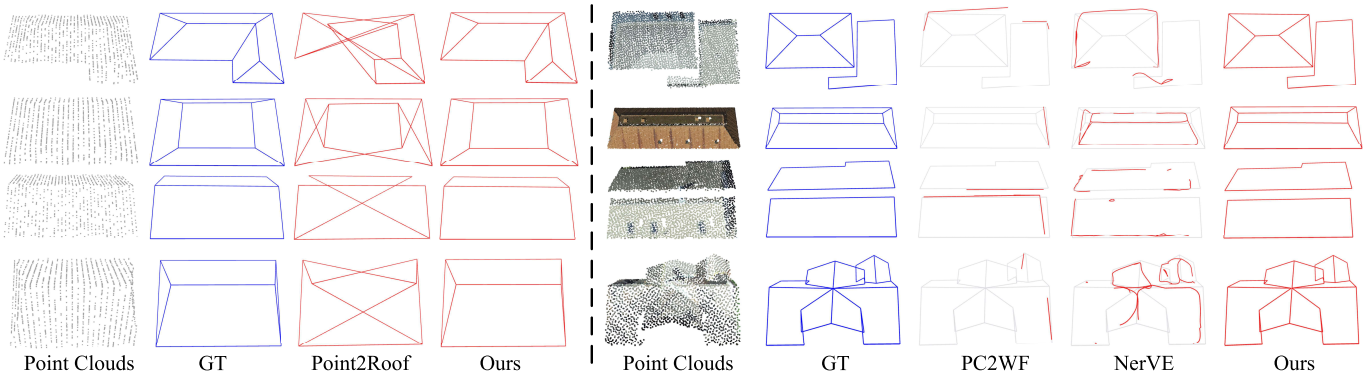


Fig. 12. Visualization experiment results of wireframe.

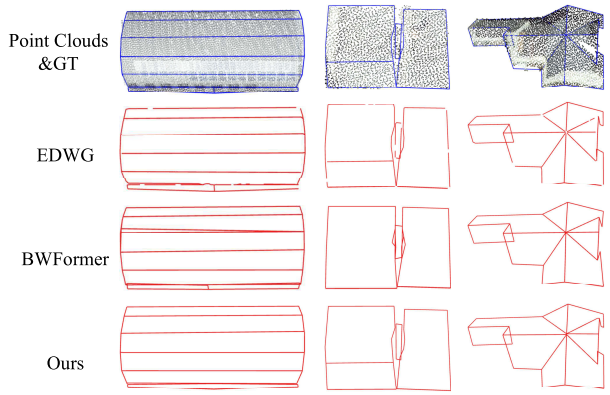


Fig. 13. Visualization experiment results of wireframe with EDWG and BWFormer.

and Saltbox. However, as the complexity of roof segment intersections increases, as in the case of Hip and Valley roofs, the detection performance inevitably declines. It is worth noting that the proposed center displacement module effectively aggregates segmentation results into localized regions, thereby facilitating clearer differentiation among them and, to some extent, mitigating the impact of edge detection errors on wireframe generation and roof reconstruction.

2) Corner Detection: To ensure a fair comparison of corner point detection performance, we conducted comparative experiments on the RoofN3D dataset, which is also used by RoofVE [49] and Point2Roof [50]. Both RoofVE and Point2Roof adopt similar strategies, performing roof corner point detection. Although our method does not include a dedicated corner detection module, we evaluate the corner detection performance by comparing the endpoints of the generated wireframes. The results of the experiments are shown in Fig. 11 and Table II.

Since RoofVE relies on voxelization of point clouds, it is inevitably subject to the limitations introduced by this process. Voxelization performs poorly on sparse LiDAR point clouds, often leading to inaccurate corner localization or missed detections. Both Point2Roof and our method are based on deep learning and achieve high corner detection accuracy after extensive training. However, Point2Roof takes only 1024 input points, making it difficult to handle more complex point cloud structures. Moreover, the roof shapes in the RoofN3D dataset are relatively limited, resulting in poor generalization performance of the trained model.

TABLE III
COMPARISON EXPERIMENTAL RESULTS OF WIREFRAME

Dataset	Method	WED↓	CD↓	HD↓	Time(s)↓
RoofN3d	Point2Roof	0.374	0.042	0.498	**0.146**
	Ours	**0.167**	**0.017**	**0.166**	0.218
Building3D-edge	PC2WF	0.621	0.174	0.857	3.659
	NerVE	-	0.059	0.681	2.522
	EDWG	-	0.047	0.504	2.845
	BWFormer	0.238	0.031	0.383	0.409
	Ours	**0.189**	**0.021**	**0.254**	**0.325**

3) Wireframe Generation: We conducted comparative experiments on wireframe generation with the Point2Roof [50] on the RoofN3D dataset and with the PC2WF [2], NerVE [35], edge detection and wireframe generation (EDWG) [30], and BWFormer [39] on the Building3D-edge dataset. The comparative visualization results are presented in Fig. 12. Point2Roof demonstrates high accuracy in corner point detection. However, it predicts connecting edges between candidate corner point pairs using the paired point attention (PPA) module. A single connection error can lead to significant deviations in the overall roof wireframe topology, thereby substantially degrading the geometric quality and practical usability of the final model. Due to its heavy reliance on the accuracy of individual edge predictions, the overall wireframe generation process lacks sufficient robustness.

NerVE is primarily designed for handling CAD point cloud data, and its applicability to LiDAR point clouds is consequently reduced. A key limitation of NerVE lies in its reliance on voxelized point clouds for edge detection, which leads to significant performance degradation when processing sparse and unevenly distributed LiDAR point clouds. Voxelization introduces resolution constraints that cause loss of detail and inaccurate edge localization. This is especially problematic in sparse regions, where voxelization can result in gaps or incorrect neighborhood relationships, thereby adversely affecting edge detection quality. Our method has the advantage of directly segmenting edges within point clouds, enabling independent fitting for each segmented region. By adopting this region-wise fitting strategy, we can effectively preserve

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

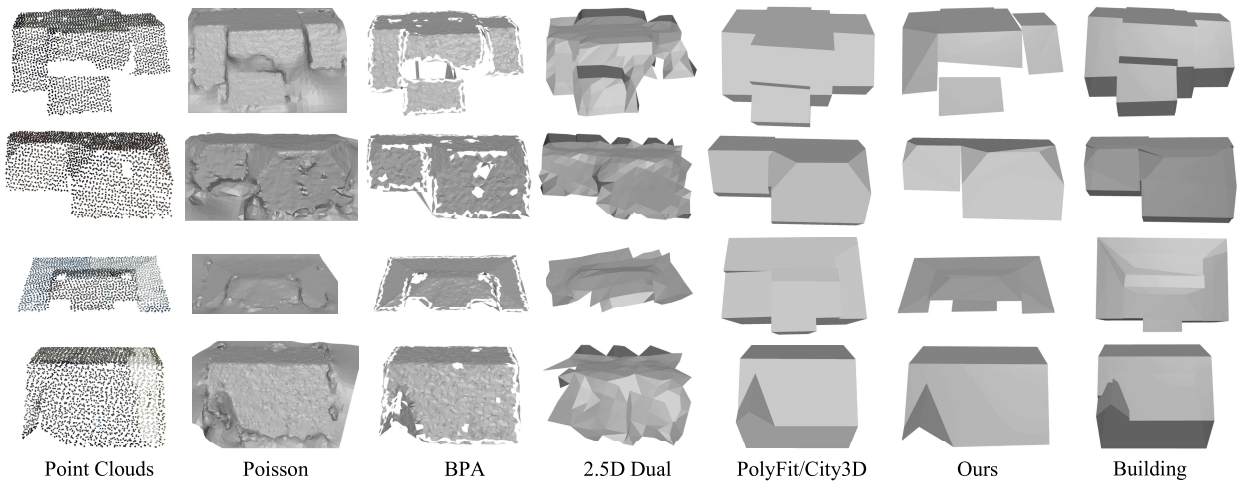


Fig. 14. Visualization experiment results of roof mesh.



Fig. 15. Visualization of Tallinn city data roof reconstruction results.

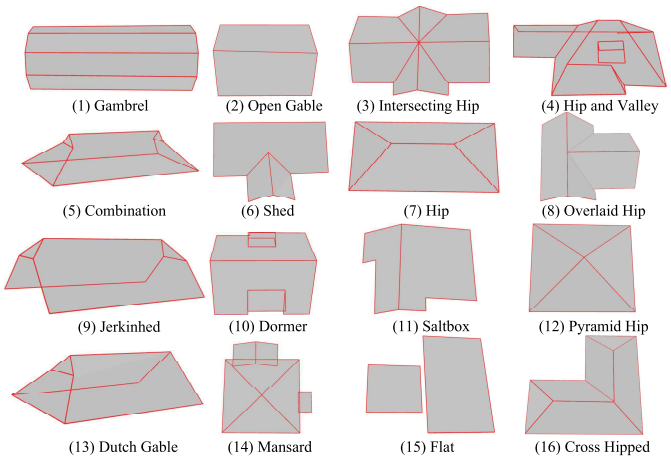


Fig. 16. Reconstruction results of 16 typical roof types.

the detailed characteristics of edges, significantly improving the accuracy and quality of wireframe generation.

As illustrated in Fig. 13, EDWG does not take noise into account, making it highly susceptible to noise interference and often resulting in broken wireframes. In contrast, the core idea of BWFormer is to project the point cloud onto a 2-D plane, predict potential 2-D corners, and then backproject them to generate 3-D corners. Experimental results demonstrate that BWFormer significantly outperforms EDWG. However, due to the limitations of the projection strategy, information loss during the transformation between 2-D and 3-D spaces is inevitable, which may lead to detail loss in certain overlapping regions. The quantitative result is reported in Table III.

In terms of runtime efficiency, our method is slightly inferior to Point2Roof in corner detection and wireframe generation tasks. This is primarily because Point2Roof employs a smaller network with fewer parameters and is mainly trained on synthetic datasets. Consequently, although it demonstrates superior efficiency, the quality of reconstruction on real airborne LiDAR point clouds remains limited.

4) Mesh Reconstruction: We visually compared the proposed method with 2.5-D dual contouring [56], PolyFit-based City3D [3], [57], and two classical reconstruction methods (Poisson [58] and ball pivoting algorithm (BPA) [59]), as shown in Fig. 14. It is worth noting that the Building3D dataset does not provide GT specifically for roof reconstruction. As a result, we use the complete building models as a qualitative reference in our visual comparisons. The 2.5-D dual contouring reconstructs 2.5-D building models from airborne LiDAR point clouds by jointly optimizing the 2-D building boundary and the 3-D surface through quadratic fitting errors. However, it may struggle to represent complex roof geometries and fine structural details due to its inherent dimensionality limitation. PolyFit-based City3D primarily focuses on planar primitives and their intersections and combinations, which makes it well-suited for relatively simple roof structures. For more complex or high-density point clouds, the number of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

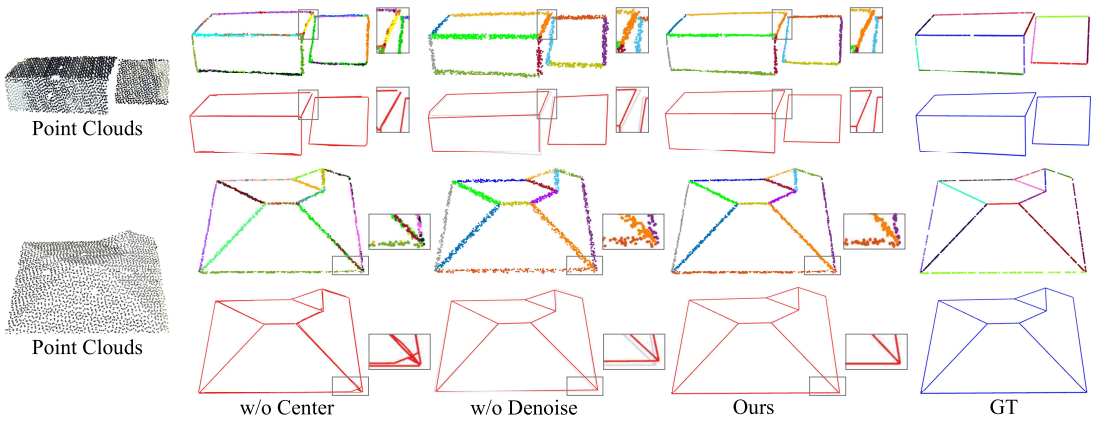


Fig. 17. Visualization results of ablation studies on different modules.

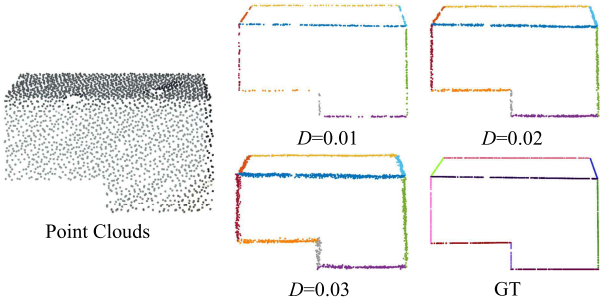


Fig. 18. Visualization results of different thresholds D.

TABLE IV
QUANTITATIVE RESULTS OF ABLATION STUDY ON DIFFERENT PARAMETERS AND MODULES

Method	WED↓	CD↓	HD↓	Acc↑	Prec↑	Rec↑
$D = 0.01$	0.204	0.027	0.368	0.944	0.853	0.915
$D = 0.03$	0.196	0.032	0.286	0.953	0.864	0.924
w/o Center	0.264	0.050	0.492	0.917	0.803	0.896
w/o Denoise	0.217	0.032	0.379	0.949	0.861	0.933
Ours	**0.189**	**0.021**	**0.254**	**0.957**	**0.868**	**0.940**

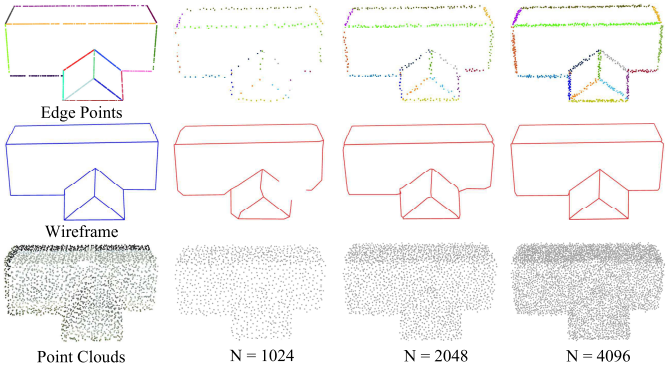


Fig. 19. Robustness studies on different numbers of point clouds.

candidate faces generated through planar intersections may increase substantially, which can pose challenges for achieving fully detailed reconstructions in certain areas.

Poisson reconstruction fits the input point cloud by solving a global implicit function, enabling the generation of smooth and continuous triangle meshes with strong watertightness and robustness. It is particularly suitable for dense and uniformly distributed point clouds. However, this method tends to over-smooth fine details and often produces spurious structures or blurs real boundaries in edge regions, making it unsuitable for reconstruction tasks that require the preservation of sharp features, such as building roof edges.

The BPA, based on triangulation principles, generates triangle meshes by rolling a virtual ball over the point cloud, which allows for better preservation of local geometric features and edge structures. It performs well when handling models with clear boundaries. However, its performance is highly sensitive to point cloud density and normal estimation. In cases of sparse or irregularly distributed point clouds, BPA is prone to generating incomplete meshes or fragmented surfaces, and it lacks guarantees for global consistency.

Benefiting from the high-quality and continuous roof boundaries obtained in our wireframe generation stage, our method demonstrates significant advantages in the roof reconstruction task. Since most building roofs are composed of multiple planar surfaces and curved structures are relatively rare, we can efficiently reconstruct the roof geometry by performing planar fitting within the polygonal regions enclosed by the wireframe. This strategy enables accurate and structurally consistent reconstruction results. The visualization results of our method applied to large-scale roof reconstruction are presented in Figs. 15 and 16.

D. Ablation Studies

Our method consists of two key parameters and modules. First, a center displacement module is introduced to aggregate edge points toward their local centers, thereby facilitating more effective differentiation of edge regions. Second, a denoise module is designed to mitigate the impact of noise. To validate the effectiveness and necessity of each component, we conducted ablation studies, demonstrating the individual contributions of each module to the overall performance. As shown in Fig. 17, the absence of the center displacement module significantly affects the completeness of the generated wireframe. This is mainly because the number of segmentation

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

regions varies across different roof LiDAR point clouds, making it difficult to achieve precise segmentation using point cloud features alone. With the center offset module, clustering can be applied to further refine the initial segmentation and eliminate misclassified points, thereby effectively improving both accuracy and precision. In contrast, the denoise module has a relatively smaller impact on the final results. Noise primarily affects the accuracy of line fitting, leading to deviations in the fit structures. Thus, its contribution is mainly reflected in improvements to distance-based metrics such as CD and HD while having limited influence on classification metrics such as Acc, Prec, and Rec.

In the data preprocessing stage, we introduce a threshold D to determine the spatial distribution range of edge points. It is worth emphasizing that our method does not rely heavily on the choice of D. As can be seen from Fig. 18, the value of D primarily affects the number of detected edge points; a larger D yields more edge points, while a smaller D results in fewer. Although the number of edge points can influence the accuracy of the final wireframe to some extent, its impact on the subsequent line fitting process is negligible. Experimental results suggest that setting $D = 0.02$ is a reasonable choice, as it does not significantly increase computational cost nor degrade the quality of the reconstructed wireframe. The results of the ablation study are presented in Table IV.

E. Robustness Studies

To further evaluate the robustness of our method, we conducted experiments on point clouds with varying point counts (1024, 2048, and 4096). The experimental results show that in the extremely sparse case of 1024 points, the method exhibited wireframe discontinuities and edge omissions in certain regions, indicating that overly sparse inputs can affect the completeness of edge structure reconstruction. However, considering that this scenario involves significant information loss compared to the original input of 10 000 points, such performance degradation is expected. Overall, although the model shows some fluctuations as the point density decreases, it still maintains reasonable structural reconstruction capability and robustness in most cases. The visualization results are shown in Fig. 19.

V. Conclusion

We propose a novel roof reconstruction method, RR-Net, which recovers 3-D roof structures from airborne LiDAR point clouds through edge segmentation and wireframe generation. The core idea of RR-Net is the joint implementation of edge segmentation and denoising, providing more accurate geometric guidance for wireframe extraction and mesh reconstruction. Based on this design, RR-Net effectively mitigates the error accumulation commonly observed in previous methods that rely heavily on corner detection and connection prediction.

Nevertheless, RR-Net still has certain limitations. First, its main challenge in handling different roof types lies in its difficulty in dealing with roofs that contain curved structures. Since RR-Net has not been specifically modeled or optimized for curved roofs during training and inference, when roof geometries rely on continuous curves for description, the line-segment-based segmentation and reconstruction mechanism often fails to accurately capture their true characteristics. Second, when the roof structure becomes overly complex, it typically involves a large number of line segments or closely arranged segments, which significantly increases the likelihood of segmentation and clustering errors. In future work, we will further explore diversified curve representations and conduct more in-depth research on the modeling and optimization of fine-grained features in complex roofs.

References

[1] S. Xia, D. Chen, R. Wang, J. Li, and X. Zhang, "Geometric primitives in LiDAR point clouds: A review," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 685–707, 2020.

[2] Y. Liu, S. D'Aronco, K. Schindler, and J. D. Wegner, "PC2WF: 3D wireframe reconstruction from raw point clouds," in *Proc. Int. Conf. Learn. Represent.*, Apr. 2021, pp. 1–7.

[3] J. Huang, J. Stoter, R. Peters, and L. Nan, "City3D: Large-scale building reconstruction from airborne LiDAR point clouds," *Remote Sens.*, vol. 14, no. 9, p. 2254, May 2022.

[4] T. Jiang et al., "Extracting 3-D structural lines of building from ALS point clouds using graph neural network embedded with corner information," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5702528.

[5] E. Maltezos, A. Doulamis, N. Doulamis, and C. Ioannidis, "Building extraction from LiDAR data applying deep convolutional neural networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 1, pp. 155–159, Jan. 2019.

[6] W. Li et al., "Weakly supervised 3-D building reconstruction from monocular remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5615315.

[7] A. Sampath and J. Shan, "Segmentation and reconstruction of polyhedral building roofs from aerial LiDAR point clouds," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 3, pp. 1554–1567, Mar. 2010.

[8] R. C. Dos Santos and M. Galo, "Robust building detection in urban environments from airborne LiDAR data: A geometry-based approach," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 9429–9441, 2024.

[9] Y. Lin et al., "Line segment extraction for large scale unorganized point clouds," *ISPRS J. Photogramm. Remote Sens.*, vol. 102, pp. 172–183, Apr. 2015.

[10] P. Tian, X. Hua, W. Tao, and M. Zhang, "Robust extraction of 3D line segment features from unorganized building point clouds," *Remote Sens.*, vol. 14, no. 14, p. 3279, Jul. 2022.

[11] Y. Lin, C. Wang, B. Chen, D. Zai, and J. Li, "Facet segmentation-based line segment extraction for large-scale point clouds," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 9, pp. 4839–4854, Sep. 2017.

[12] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 652–660.

[13] C. R. Qi, Y. Li, H. Su, and L. Guibas, "PointNet+: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Adv. Neural Inf. Process. Syst.*, Oct. 2017, pp. 1–8.

[14] R. Wang, S. Huang, and H. Yang, "Building3D: A urban-scale dataset and benchmarks for learning roof structures from point clouds," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2023, pp. 20076–20086.

[15] T. Hackel, N. Savinov, L. Ladicky, J. D. Wegner, K. Schindler, and M. Pollefeys, "SEMANTIC3D.NET: A new large-scale point cloud classification benchmark," *ISPRS Ann. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 4, pp. 91–98, May 2017.

[16] L. Cao, Y. Xu, J. Guo, and X. Liu, "WireframeNet: A novel method for wireframe generation from point cloud," *Comput. Graph.*, vol. 115, pp. 226–235, Oct. 2023.

[17] Y. Chen, S. Zhang, T. Han, Y. Du, W. Zhang, and J. Li, "Chat3D: Interactive understanding 3D scene-level point clouds by chatting with foundation model for urban ecological construction," *ISPRS J. Photogramm. Remote Sens.*, vol. 212, pp. 181–192, Jun. 2024.

[18] Y. Li, S. Liu, X. Yang, J. Guo, J. Guo, and Y. Guo, "Surface and edge detection for primitive fitting of point clouds," in *Proc. Special Interest Group Comput. Graph. Interact. Techn. Conf. Conf.*, Jul. 2023, pp. 1–10.

[19] Y. Liu, J. Chen, S. Pan, D. Cohen-Or, H. Zhang, and H. Huang, "Split-and-fit: Learning B-reps via structure-aware Voronoi partitioning," *ACM Trans. Graph.*, vol. 43, no. 4, pp. 1–13, Jul. 2024.

[20] C. Li, H. Pan, A. Bousseau, and N. J. Mitra, "Free2CAD: Parsing freehand drawings into CAD commands," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 1–16, Jul. 2022.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.

[21] S. Xia and R. Wang, "A fast edge extraction method for mobile LiDAR point clouds," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 8, pp. 1288–1292, Aug. 2017.

[22] T. Hackel, J. D. Wegner, and K. Schindler, "Contour detection in unstructured 3D point clouds," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 1610–1618.

[23] S. Di et al., "PECo: A point-edge collaborative framework for global-aware urban building contouring from unstructured point clouds," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5700719.

[24] C.-E. Himeur, T. Lejemble, T. Pellegrini, M. Paulin, L. Barthe, and N. Mellado, "PCEDNet: A lightweight neural network for fast and interactive edge detection in 3D point clouds," *ACM Trans. Graph.*, vol. 41, no. 1, pp. 1–21, Nov. 2021.

[25] Y. Zang et al., "LCE-NET: Contour extraction for large-scale 3-D point clouds," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5704413.

[26] S. Koch et al., "ABC: A big CAD model dataset for geometric deep learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 9593–9603.

[27] H. Huang, S. Wu, M. Gong, D. Cohen-Or, U. Ascher, and H. Zhang, "Edge-aware point set resampling," *ACM Trans. Graph.*, vol. 32, no. 1, pp. 1–12, Feb. 2013.

[28] L. Yu, X. Li, C. Fu, D. Cohen–Or, and P. Heng, "EC-Net: An edge-aware point set consolidation network," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2018, pp. 386–402.

[29] X. Wang et al., "PIE-NET: Parametric inference of point cloud edges," in *Proc. Adv. Neural Inf. Process. Syst.*, Oct. 2020, pp. 20167–20178.

[30] Y. Xu, J. Guo, L. Cao, and X. Liu, "EDWG: Efficient edge detection and wireframe generation from point clouds," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–12, 2025.

[31] T. Zhao, M. Yu, P. Alliez, and F. Lafarge, "Sharp feature consolidation from raw 3D point clouds via displacement learning," *Comput. Aided Geometric Design*, vol. 103, Jun. 2023, Art. no. 102204.

[32] X. Jiao et al., "MSL-Net: Sharp feature detection network for 3D point clouds," *IEEE Trans. Vis. Comput. Graph.*, vol. 30, no. 9, pp. 6433–6446, Sep. 2024.

[33] W. Zhang, L. Chen, Z. Xiong, Y. Zang, J. Li, and L. Zhao, "Large-scale point cloud contour extraction via 3D guided multi-conditional generative adversarial network," *ISPRS J. Photogramm. Remote Sens.*, vol. 164, pp. 97–105, Jun. 2020.

[34] X. Xin, W. Huang, S. Zhong, M. Zhang, Z. Liu, and Z. Xie, "Accurate and complete line segment extraction for large-scale point clouds," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 128, Apr. 2024, Art. no. 103728.

[35] X. Zhu, D. Du, W. Chen, Z. Zhao, Y. Nie, and X. Han, "NerVE: Neural volumetric edges for parametric curve extraction from point cloud," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13601–13610.

[36] H. Yang, S. Huang, R. Wang, and X. Wang, "Self-supervised pre-training for 3-D roof reconstruction on LiDAR data," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.

[37] S. Huang, R. Wang, B. Guo, and H. Yang, "PBWR: Parametric-building-wireframe reconstruction from aerial LiDAR point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 27778–27787.

[38] Y. Liu, R. Wang, S. Huang, and G. Cai, "EdgeDiff: Edge-aware diffusion network for building reconstruction from point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 17008–17018.

[39] Y. Liu et al., "BWFormer: Building wireframe reconstruction from airborne LiDAR point cloud with transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 22215–22224.

[40] X. Ma, Y. Liu, W. Zhou, R. Wang, and H. Huang, "Generating 3D house wireframes with semantics," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2024, pp. 223–240.

[41] V. Verma, R. Kumar, and S. Hsu, "3D building detection and modeling from aerial LiDAR data," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 2, Oct. 2006, pp. 2213–2220.

[42] A. Henn, G. Gröger, V. Stroh, and L. Plümer, "Model driven reconstruction of roofs from sparse LiDAR point clouds," *ISPRS J. Photogramm. Remote Sens.*, vol. 76, pp. 17–29, Feb. 2013.

[43] H. Huang, C. Brenner, and M. Sester, "A generative statistical approach to automatic 3D building roof reconstruction from laser scanning data," *ISPRS J. Photogramm. Remote Sens.*, vol. 79, pp. 29–43, May 2013.

[44] B. Xiong, M. Jancosek, S. O. Elberink, and G. Vosselman, "Flexible building primitives for 3D building modeling," *ISPRS J. Photogramm. Remote Sens.*, vol. 101, pp. 275–290, Mar. 2015.

[45] L. Li et al., "A boundary-aware point clustering approach in Euclidean and embedding spaces for roof plane segmentation," *ISPRS J. Photogramm. Remote Sens.*, vol. 218, pp. 518–530, Dec. 2024.

[46] R. Cao, Y. Zhang, X. Liu, and Z. Zhao, "3D building roof reconstruction from airborne LiDAR point clouds: A framework based on a spatial database," *Int. J. Geographical Inf. Sci.*, vol. 31, no. 7, pp. 1359–1380, Jul. 2017.

[47] Y. Chen, L. Cheng, M. Li, J. Wang, L. Tong, and K. Yang, "Multiscale grid method for detection and reconstruction of building roofs from airborne LiDAR data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 10, pp. 4081–4094, Oct. 2014.

[48] H. Yang, S. Huang, and R. Wang, "Efficient roof vertex clustering for wireframe simplification based on the extended multiclass twin support vector machine," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.

[49] G. Kong, Y. Zhao, and H. Fan, "Detecting vertices of building roofs from ALS point cloud data," *Int. J. Digit. Earth*, vol. 16, no. 2, pp. 4811–4830, Nov. 2023.

[50] L. Li et al., "Point2Roof: End-to-end 3D building roof modeling from airborne LiDAR point clouds," *ISPRS J. Photogramm. Remote Sens.*, vol. 193, pp. 17–28, Nov. 2022.

[51] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph CNN for learning on point clouds," *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, Oct. 2019.

[52] S. Chen, J. Fang, Q. Zhang, W. Liu, and X. Wang, "Hierarchical aggregation for 3D instance segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 15467–15476.

[53] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 318–327, Feb. 2020.

[54] G. Sharma, D. Liu, S. Maji, E. Kalogerakis, S. Chaudhuri, and R. Měch, "ParSeNet: A parametric surface fitting network for 3D point clouds," in *Proc. Eur. Conf. Comput. Vis.*, Aug. 2020, pp. 261–276.

[55] A. Wichmann, A. Agoub, and M. Kada, "RoofN3D: Deep learning training data for 3D building reconstruction," *Int. Arch. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 42, pp. 1191–1198, May 2018.

[56] Q.-Y. Zhou and U. Neumann, "2.5D dual contouring: A robust approach to creating building models from aerial LiDAR point clouds," in *Proc. Eur. Conf. Comput. Vis.*, May 2010, pp. 115–128.

[57] L. Nan and P. Wonka, "PolyFit: Polygonal surface reconstruction from point clouds," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2353–2361.

[58] M. Kazhdan and H. Hoppe, "Screened Poisson surface reconstruction," *ACM Trans. Graph.*, vol. 32, no. 3, pp. 1–13, Jul. 2013.

[59] F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, and G. Taubin, "The ball-pivoting algorithm for surface reconstruction," *IEEE Trans. Vis. Comput. Graph.*, vol. 5, no. 4, pp. 349–359, Oct. 1999.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:57:07 UTC from IEEE Xplore. Restrictions apply.